

# Property-driven Parallel Symbolic Model Checking of LTL

Yuheng Su<sup>†‡</sup>, Yingcheng Li<sup>†‡</sup>, Qiusong Yang<sup>†§</sup>, Yiwei Ci<sup>‡</sup>, Ziyu Huang<sup>¶</sup>

<sup>†</sup>University of Chinese Academy of Sciences

<sup>‡</sup>Institute of Software, Chinese Academy of Sciences

<sup>§</sup>Laboratory for Advanced Computing and Intelligence Engineering, Wuxi, China

<sup>¶</sup>Beijing Forestry University

**Abstract**—Model checking is an automated method used to formally verify systems by checking them against properties. However, a major problem in model checking is the state explosion. To overcome this challenge, one approach is to utilize parallel processing capabilities to either speed up computations or handle larger-scale problems. Explicit model checking has lower computational complexity and can be easily parallelized. There are numerous parallel explicit model checking algorithms available in the literature. Symbolic model checking offers significant advantages over explicit model checking in terms of problem scalability and verification speed. However, treating states encountered during the search as sets poses a challenge in devising efficient parallel algorithms. As a result, current research on parallelizing symbolic model checking has primarily focused on reachability analysis or safety properties, rather than attempting to parallelize the nested fixpoint calculations. In this paper, we propose a novel property-driven approach for parallel symbolic model checking of full LTL. Our algorithm introduces a fair model state labelling function that forms a partition of the nested fixpoint across the product combining the model and the property Büchi automaton. The experimental results demonstrate significant speedup, ranging from 2.81 to 17.19 times compared to sequential approaches on a 32-core machine. Moreover, in comparison to existing parallel model checking methods, our approach not only surpasses those relying on BDD libraries with a maximum improvement of up to 134% and an average improvement of 33.1% but also demonstrates significant superiority over the state-of-the-art parallel explicit model checker.

## I. INTRODUCTION

*Model checking* is an automated method for formally verifying systems against temporal properties [1]. However, the state space must be exhaustively explored either explicitly or symbolically, leading to the state explosion problem that has plagued model checking since its inception [2]. To address this issue, researchers have developed clever methods such as abstractions [3] and symbolic representations [4]. Another approach is to leverage the rapid development of computing, storage, and communication technologies by parallelizing sequential model checking using multi-core [5], distributed systems [6], GPGPU [7], and other modes of computation.

*Explicit model checking* is amenable to parallelization, and extensive parallel algorithms exist in the literature. In explicit model checking, each state encountered during BFS, DFS, or some variants is explicitly stored and individually manipulated. Explicit model checking algorithms based on BFS or the detection of strongly connected components (SCC) belong to the Nick class [8], which theoretically have highly parallel algorithms [9]. Although DFS is inherently sequential, and thus the classic nested DFS model checking algorithm [10] based on finding an accepting and fair cycle does not belong to the Nick class, there are also many efficient parallel explicit model checking algorithms based on DFS [8].

*Symbolic model checking* offers significant advantages, compared to explicit model checking, in terms of the scale of problems and verification speed. Symbolic model checking uses characteristic functions to represent a set of states. Symbolic representations, such as BDDs [11], are normally more compact than explicit representations,

making it possible to verify problems with larger scales. Modern symbolic model checking algorithms are variants of the Emerson-Lei algorithm [12], which includes nested fixpoint calculations.

However, treating states encountered as sets poses a challenge in devising efficient parallel symbolic algorithms. The most challenging task lies in devising an efficient algorithm for computing the nested fixpoint when the symbolic state space is divided into slices. As a result, current research on parallelizing symbolic model checking has primarily focused on reachability analysis or safety properties, avoiding the parallelization of nested fixpoint calculations. Building upon distributedly partitioned BDDs, a parallel symbolic model checking algorithm for safety properties was introduced in [13]. There have been also various studies on parallel symbolic model checking of systems with specific structures. For instance, research has been conducted on the reachability analysis of security protocols with a bounded number of sessions [14] and the verification of split-invariants for asynchronously composed processes communicating through shared-memory [15]. Unlike these studies, which do not focus on full model checking, [16] proposed a symbolic algorithm for the model checking of propositional  $\mu$ -calculus based on distributively evaluating subformulas. However, formulas to be verified are generally not large in scale, the degree of parallelism is limited.

In this paper, we present property-driven parallel symbolic model checking of full LTL. The contributions is summarized as follows:

- We propose a property-driven state space partition method for nested fixpoint computation. This algorithm utilizes a fair model state labelling function that ensures the labeling of a model state  $s$  in an automaton state  $q$  if and only if there exists a fair path starting from  $(s, q)$  in the product. The fair model state labelling function naturally forms a partition of the nested fixpoint. Therefore, by computing the labelling function, we effectively implement the symbolic model checking of LTL.
- We propose a parallelize algorithm to compute the state labelling function. This algorithm takes advantage of the labelling function can be computed using a backward BFS manner, which is well-suited for parallelization. In our approach, the computation is performed by a group workers. The number of workers used in this process is equal to the number of states in the automaton, denoted as  $n$ . However, we have also devised mechanisms to increase the degree of parallelization beyond  $n$ , allowing for more workers to be utilized. To ensure efficient usage of resources, we have also introduced load balancing techniques.
- We conduct a comprehensive evaluation of our parallel algorithm and compared it to existing approaches. The results highlight the effectiveness and efficiency of our proposed method. The experimental results on a 32-core machine demonstrate significant speedup, ranging from 2.81 to 17.19 times compared to sequential approaches. Moreover, in comparison to existing parallel methods, our approach not only surpasses those relying on BDD libraries with a maximum improvement of up to 134%

\* Qiusong Yang is the corresponding author.

and an average of 33.1% but also demonstrates superiority over the parallel explicit model checker LTSMin [5].

## II. PRELIMINARIES

**Kripke structure:** Without loss of generality, we assume the system is modelled as Kripke structure. Specifically, a Kripke structure is represented by a tuple  $M = (S, S_0, R, AP, L)$  in which  $S$  denotes the set of states,  $S_0 \subseteq S$  represents the set of initial states,  $R \subseteq S \times S$  denotes the transition relation,  $AP$  is the set of atomic propositions, and  $L : S \rightarrow 2^{AP}$  is a function that labels each state with the set of atomic propositions that are true in that state [1]. A path of length  $l$  is defined as a sequence of states  $s_0, \dots, s_{l+1}$  where  $s_0 \in S_0$  and for all  $0 \leq i \leq l$ ,  $(s_i, s_{i+1}) \in R$ .

**Linear temporal logic (LTL)** is a formalism for specifying temporal properties derived from Kripke structures. It extends propositional logic with temporal operators such as **X** (next), **F** (eventually), **G** (globally), and **U** (until), allowing the expression of time-dependent behaviors [1]. The properties expressed in LTL can be classified into safety and liveness.

**Büchi automaton:** LTL model checking typically involves negating the formula, converting it to a Büchi automaton, and checking the emptiness of its product with the Kripke model. As the Büchi automaton generated from an LTL formula normally has an alphabet over the power set of  $AP$  and includes atomic propositions as labels in each state, in addition to the standard definition, we have incorporated a state labeling function in the following definition. A Büchi automaton is defined as  $\mathcal{B} = (\Sigma, Q, \Delta, Q_0, \mathcal{L}, F)$ , where  $\Sigma = 2^{AP}$  is the finite alphabet,  $Q$  is the finite set of states,  $\mathcal{L} : Q \rightarrow 2^{AP}$  is the state labelling function,  $\Delta \subseteq Q \times \Sigma \times Q$  is the transition relation such that if  $(q, \sigma, q') \in \Delta$  then  $\sigma = \mathcal{L}(q)$ ,  $Q_0$  is the set of initial state, and  $F = \{P_1, P_2, \dots, P_k\}$  where for every  $1 \leq i \leq k$ ,  $P_i \subseteq Q$ . Similarly, an infinite path  $\rho$  is a sequence states  $q_0, \dots, q_i, \dots$  such that  $q_0 \in Q_0$  and, for all  $i \geq 0$ ,  $(q_i, \nu(i), q_{i+1}) \in \Delta$ , where  $\nu(i)$  denotes the  $i$ -th input letter of the word sequence  $\nu \in \Sigma^*$ . A Büchi automaton accepts only infinite *illustrate* paths in which there exist at least one state from each  $P_i$  that appears infinitely often. Given a Kripke structure  $M = (S, S_0, R, AP, L)$  and a Büchi automaton  $\mathcal{B} = (2^{AP}, Q, \Delta, Q_0, \mathcal{L}, F)$ , their synchronized product is denoted as  $M \otimes \mathcal{B} = (2^{AP}, Q_\otimes, \Delta_\otimes, Q_{\otimes 0}, F_\otimes)$ . Here,  $Q_\otimes = S \times Q$ ,  $Q_{\otimes 0} = S_0 \times Q_0$ , and  $((s_1, q_1), \sigma, (s_2, q_2)) \in \Delta_\otimes$  if and only if  $(s_1, s_2) \in R$ ,  $L(s_1) = \mathcal{L}(q_1)$ , and  $(q_1, \sigma, q_2) \in \Delta$ . Furthermore,  $F_\otimes(s_1, q_1) = F(q_1)$ .

## III. PROBLEM FORMULATION

The formula  $\mathbf{EG}\varphi$  expresses the existence of a path in which  $\varphi$  holds globally. When fairness is considered, the set of states satisfying  $\mathbf{EG}\varphi$  can be obtained through the following nested fixpoint:

$$[\mathbf{EG}\varphi]_M^{\text{Fair}} = \nu Z. \varphi \wedge \bigwedge_{k=1}^n \mathbf{EXE}(\varphi \mathbf{U}(Z \wedge P_k)) \quad (1)$$

Here,  $\nu$  represents the greatest fixpoint, and  $\{P_1, \dots, P_n\}$  represents the fairness. If any of the initial states of  $M$  belongs to  $[\mathbf{EG}\varphi]_M^{\text{Fair}}$ , then  $M \models \mathbf{EG}\varphi$ . Consequently, the formula  $\mathbf{EG}true$  characterizes the set of states from each of which there exists a fair path.

The symbolic model checking of LTL based on the computation of  $\mathbf{EG}true$ , assuming  $F = P_1$  without loss of generality, is presented in Algorithm 1. When the fixpoint is reached, only the states from which there exists a fair path are kept. The procedure above heavily relies on the use of symbolic state representation, utilizing sets of states in every iteration. This approach significantly enhances its effectiveness and clarity. However, it also makes decision procedures employed

## Algorithm 1: Symbolic algorithm for LTL model checking

**Input:** a Kripke structure  $M$ , a Büchi  $\mathcal{B}_{\neg\varphi}$  automaton corresponding to the LTL formula to be verified.  
**Output:** a boolean result indicating whether the language accepted by  $M \otimes \mathcal{B}_{\neg\varphi}$  is empty.

```

1 begin /* Compute  $\mathbf{EG}true = \nu Z. \mathbf{EXE}(Z \wedge F_\otimes)$  */
2    $S_1 \leftarrow S \times Q$ ,  $Q' \leftarrow S_1$ ;
3   do
4      $Q \leftarrow Q'$ ;
5     begin /*  $\mathbf{EF}(Q \wedge F_\otimes) = \mu Z. (Q \wedge F_\otimes) \vee \mathbf{EX}Z$  */
6        $W' \leftarrow \emptyset$ ;
7       do
8          $W \leftarrow W'$ ;
9          $W' \leftarrow (Q \wedge F_\otimes) \vee \text{Pre}_\otimes(W)$ ;
10      while  $W \neq W'$ ;
11     begin /* Compute  $\mathbf{EX}$  */
12        $Q' \leftarrow \text{Pre}_\otimes(Q)$ ;
13   while  $Q \neq Q'$ ;
14 begin /* Check emptiness */
15   if  $\exists (s_0, q_0) \in S_0 \otimes Q_0$  such that  $(s_0, q_0) \in Q$  then
16     return false
17   else
18     return true

```

in explicit parallel model checking, such as loop detection based on nested DFS [10], or loop detection based on BFS [17], impractical.

As discussed previously, the current research on parallelizing symbolic model checking primarily focuses on reachability analysis and safety properties, while neglecting the parallelization of nested fixpoint calculations. Therefore, there is a gap in this research area as no investigations have been conducted on parallelizing the nested fixpoint calculation process. Our paper aims to present an effective approach for parallelizing the nested fixpoint computation process, enabling parallel symbolic model checking of full LTL.

## IV. PROPERTY-DRIVEN PARALLEL SYMBOLIC MODEL CHECKING

The term *property-driven* refers to the transformation of nested fixpoint computation into the parallel computation of  $n$  smaller fixpoints, where  $n$  is the number of states of the property automaton.

### A. Sequential algorithm

Given a Kripke structure  $M = (S, S_0, R, AP, L)$ , a Büchi automaton  $\mathcal{B} = (2^{AP}, Q, \Delta, Q_0, \mathcal{L}, F)$  and their synchronous product, we have the following definitions:

**Definition IV.1** (Model state labelling function). *Let  $\mathcal{L}_M$  be a function:  $Q \rightarrow 2^S$ . It should be noted that any subset of  $S$  can be characterized by a predicate over  $AP$ .*

**Definition IV.2** (Projection). *For a subset  $Q' \subseteq Q$  and a model state labelling function  $\mathcal{L}_M$ , the projection of  $\mathcal{L}_M$  over  $Q'$  is denoted as  $\mathcal{L}_{M|Q'}(q) = \mathcal{L}_M(q)$  if  $q \in Q'$ , and  $\emptyset$  otherwise.*

**Definition IV.3** (Fair model state labelling function). *A model state labelling function  $\mathcal{L}_M$  is considered fair if and only if for any state  $s$  in the set  $\mathcal{L}_M(q)$ , there exists a fair path starting from  $(s, q)$  in the product.*

**Theorem IV.1.** *For a state labeling function, the set  $\{\mathcal{L}_M(q) \mid \text{for all } q \in Q\}$  forms a partition of the nested fixpoint  $\mathbf{EG}true$ .*

**Algorithm 2:** Sequential property-driven symbolic algorithm

**Input:** a Kripke structure  $M$ ; a Büchi  $\mathcal{B}_{\neg\varphi}$  automaton corresponding to the LTL formula to be verified.  
**Output:** a boolean result indicating whether the language accepted by  $M \otimes \mathcal{B}_{\neg\varphi}$  is empty.

```

1   $\mathcal{L}_{\mathcal{M}}^0(q) \leftarrow \text{true}$ , for all  $q \in Q$ ;
2  do
3     $\mathcal{L}_{\mathcal{M}}^1 \leftarrow \mathcal{L}_{\mathcal{M}}^0$ ;
4     $\mathcal{L}_{\mathcal{M}}^0(q) \leftarrow \mathcal{L}(q) \cap \mathcal{L}_{\mathcal{M}}^1(q)$  if  $q \in P_1$ ,  $\emptyset$  otherwise;
      /*  $s \in \mathcal{L}_{\mathcal{M}}^0(q)$  if and only if  $(s, q) \in \mathbf{EF}(\mathcal{L}(q) \cap \mathcal{L}_{\mathcal{M}}^1(q))$  */
5     $\text{FIFO queue} \leftarrow P_1$ ;
6    while  $\text{queue} \neq \emptyset$  do
7       $q \leftarrow \text{queue.remove}$ ;
8      for each  $q' \in \text{Pre}_{\mathcal{B}}(q)$  do
9         $\text{update} \leftarrow \text{Pre}_{\mathcal{M}}(\mathcal{L}_{\mathcal{M}}^0(q)) \cap \mathcal{L}(q') \cap \overline{\mathcal{L}_{\mathcal{M}}^0(q')}$ ;
10       if  $\text{update} \neq \emptyset$  then
11          $\mathcal{L}_{\mathcal{M}}^0(q') = \mathcal{L}_{\mathcal{M}}^0(q') \cup \text{update}$ ;
12          $\text{add } q' \text{ to queue}$ ;
      /* Compute  $\mathbf{EX}(\mathcal{L}_{\mathcal{M}}^0)$  */
13     $\mathcal{L}_{\mathcal{M}}^{\text{EX}}(q) \leftarrow \text{false}$ , for all  $q \in Q$ ;
14     $\text{FIFO queue} \leftarrow Q$ ;
15    while  $\text{queue} \neq \emptyset$  do
16       $q \leftarrow \text{queue.remove}$ ;
17      for each  $q' \in \text{Pre}_{\mathcal{B}}(q)$  do
18         $\mathcal{L}_{\mathcal{M}}^{\text{EX}}(q') = \mathcal{L}_{\mathcal{M}}^{\text{EX}}(q') \cup (\text{Pre}_{\mathcal{M}}(\mathcal{L}_{\mathcal{M}}^0(q)) \cap \mathcal{L}(q'))$ ;
19     $\mathcal{L}_{\mathcal{M}}^0 = \mathcal{L}_{\mathcal{M}}^{\text{EX}}$ ;
20    while  $\mathcal{L}_{\mathcal{M}}^1 \neq \mathcal{L}_{\mathcal{M}}^0$ ;
      /* Check emptiness */
21    if  $\exists s_0 \in S_0, q_0 \in Q_0$  such that  $s_0 \in \mathcal{L}_{\mathcal{M}}^0(q_0)$  then
22      return false
23    else
24      return true

```

In Algorithm 2, we propose a sequential property-driven symbolic algorithm for model checking LTL. The main objective of the algorithm is to compute the fair model state labeling function. When line 4 is first executed, each accepting state  $q$  is labeled with the model states that satisfy  $\mathcal{L}(q)$ . The code between line 5 and line 12 performs backward labeling on the Büchi automaton's states. This ensures that a model state  $s$  is labeled in some state  $q$  if and only if  $(s, q) \in \mathbf{EF}(\mathcal{L}(q) \cap \mathcal{L}_{\mathcal{M}}^1(q))$ . In line 11, the code ensures that if a model state  $s'$  is added to the labeling of  $q'$ , there must exist a state  $s \in \mathcal{L}_{\mathcal{M}}^0(q)$  such that  $((s', q'), \mathcal{L}(q'), (s, q)) \in \Delta_{\otimes}$ . The subsequent steps in lines 13 to 19 calculate the **EX** operation. This operation involves removing a state's label  $s$  from some accepting state  $q$  under the following conditions: either  $(s, q)$  is not part of any loop, or it cannot reach any other accepting states.

A Kripke structure and an automaton are presented in Figure 1. Initially,  $\mathcal{L}_{\mathcal{M}}(d) = \mathcal{L}(d) = \{x, y, z\}$ ,  $\mathcal{L}_{\mathcal{M}}(a) = \mathcal{L}_{\mathcal{M}}(b) = \mathcal{L}_{\mathcal{M}}(c) = \emptyset$ , and  $\text{queue} = \{d\}$ . Then get  $d$  from the  $\text{queue}$ , and calculate the pre-image of  $\mathcal{L}_{\mathcal{M}}(d)$ ,  $\text{Pre}_{\mathcal{M}}(\mathcal{L}_{\mathcal{M}}(d)) = \{x, y, z\}$ . And then transfer it to the predecessor states in the automaton,  $\text{Pre}_{\mathcal{B}}(d) = \{b, c, d\}$ . For state  $b$ , calculate all states that belong to  $\text{Pre}_{\mathcal{M}}(\mathcal{L}_{\mathcal{M}}(d)) \cap \mathcal{L}(b)$  and do not belong to  $\mathcal{L}_{\mathcal{M}}(b)$ ,  $\text{Pre}_{\mathcal{M}}(\mathcal{L}_{\mathcal{M}}(d)) \cap \mathcal{L}(b) \cap \overline{\mathcal{L}_{\mathcal{M}}(b)} = \{x, z\}$ . If it is not empty, add these states to  $\mathcal{L}_{\mathcal{M}}(b)$  and enqueue  $b$ . The same for  $c$  and  $d$ , then  $\mathcal{L}_{\mathcal{M}}(a) = \emptyset$ ,  $\mathcal{L}_{\mathcal{M}}(b) = \{x, z\}$ ,

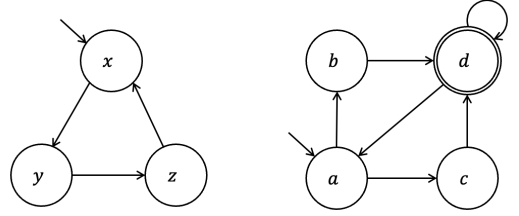


Fig. 1: A Kripke structure  $S_0 = \{x\}$ , and a Büchi automaton,  $Q_0 = \{a\}$ ,  $P_1 = \{d\}$ ,  $\mathcal{L}(a) = \{y\}$ ,  $\mathcal{L}(b) = \{x, z\}$ ,  $\mathcal{L}(c) = \{y, z\}$ ,  $\mathcal{L}(d) = \{x, y, z\}$

**Algorithm 3:** Parallel property-driven symbolic algorithm

**Input:** a Kripke structure  $M$ ; the Büchi  $\mathcal{B}_{\neg\varphi}$  automaton corresponding to  $\neg\varphi$ , where  $\varphi$  is the LTL formula to be verified. Let  $n = |Q|$ .  
**Output:** a boolean result indicating whether the language accepted by  $M \otimes \mathcal{B}_{\neg\varphi}$  is empty  
**Data:**  $\mathcal{L}_{\mathcal{M}}^0(q)$  with each element initialized to true,  $\mathcal{L}_{\mathcal{M}}^1(q)$ .

```

1  do
2     $\mathcal{L}_{\mathcal{M}}^1 = \mathcal{L}_{\mathcal{M}}^0$ ;
3    for each  $q \in Q$  do /* Compute EXEF */
4      |  $\text{Spawn Labelling}_q(\mathcal{L}_{\mathcal{M}^1 P_1}(q))$ ;
5    for each  $q \in Q$  do
6      |  $\mathcal{L}_{\mathcal{M}}^0(q) \leftarrow \text{Sync Labelling}_q$ ;
7    while  $\mathcal{L}_{\mathcal{M}}^0 \neq \mathcal{L}_{\mathcal{M}}^1$ ;
      /* Check emptiness */
8    if  $\exists s_0 \in S_0, q_0 \in Q_0$  such that  $s_0 \in \mathcal{L}_{\mathcal{M}}^0(q_0)$  then
9      | return false
10   else
11     | return true

```

$\mathcal{L}_{\mathcal{M}}(c) = \{y, z\}$ ,  $\mathcal{L}_{\mathcal{M}}(d) = \{x, y, z\}$  and  $\text{queue} = \{b, c\}$ . Continue this process until the  $\text{queue}$  becomes empty, indicating the occurrence of the fixpoint of **EF**. Then, proceed to calculate **EX** and repeat the entire process until  $\mathcal{L}_{\mathcal{M}}$  is no longer updated (line 30). Finally,  $\mathcal{L}_{\mathcal{M}}(a) = \{y\}$ ,  $\mathcal{L}_{\mathcal{M}}(b) = \{x, z\}$ ,  $\mathcal{L}_{\mathcal{M}}(c) = \{y, z\}$ ,  $\mathcal{L}_{\mathcal{M}}(d) = \{x, y, z\}$ , which is the fixpoint of **EGtrue**. The property is safe because the only initial state  $x \notin \mathcal{L}_{\mathcal{M}}(a)$ .

**Theorem IV.2.** Algorithm 2 computes the fair model state labelling function.

*Proof.* In the algorithm, a model state  $s$  is marked in a particular state  $q$  if and only if  $(s, q)$  is an accepting state of the product or can reach an accepting state within a finite number of steps. It is important to note that the algorithm will eventually terminate since the **EX** operations cannot run indefinitely. Upon termination, the nested fixpoint computation guarantees that  $s$  is marked in a state  $q$  if and only if there exists a fair path that starts from  $(s, q)$ .  $\square$

**B. Parallel algorithm**

The algorithm 2 effectively labels each Büchi state in a backward BFS manner, making it well-suited for parallelization. The parallel version is presented in Algorithm 3 and Algorithm 4.

In Algorithm 3, lines 1-7 compute the **EXEF** formula in parallel, continuing in an iterative manner until a fixed point is reached. More specifically, line 4 of the code spawns a separate worker for each state in the Büchi automaton, and each worker executes Algorithm 4. The projection of  $\mathcal{L}_{\mathcal{M}}^0$  onto  $P_1$  in line 4 will assign labels to each

state  $q$  in  $P_1$  based on the modeling states that satisfy  $q$ , leaving all other states labeled with the empty set. In line 6, the algorithm is blocked until all workers have completed their tasks, subsequently retrieving the results of their computations.

---

**Algorithm 4:** Labelling <sub>$q$</sub> 


---

**Input:**  $\mathcal{L}_{\mathcal{M}}^0(q), \mathcal{L}(q)$ .  
**Output:** a set of states indicating fair model state labelling function over  $q$

```

1  $\mathcal{L}_{\mathcal{M}}^{EF}(q) = \mathcal{L}(q) \cap \mathcal{L}_{\mathcal{M}}^0(q)$ ;
2 if  $\mathcal{L}_{\mathcal{M}}^{EF}(q) \neq \text{false}$  then
3   for each  $q' \in \text{Pre}_B(q)$  do
4     | Send EF message  $\mathcal{L}_{\mathcal{M}}^{EF}(q)$  to Labelling $q'$ ;
5 do
6   if all other Labelling $q$  blocked in receiving then
7     | Notify other Labelling $q$  stopping receiving
8     | and breaking;
9     | Break;
10  | Wait until an EF message arrives;
11  | Receive a Set message ef_message;
12  |  $\text{update} = \text{Pre}_M(\text{ef\_message}) \cap \mathcal{L}(q) \cap \overline{\mathcal{L}_{\mathcal{M}}^{EF}(q)}$ ;
13  |  $\mathcal{L}_{\mathcal{M}}^{EF}(q) = \mathcal{L}_{\mathcal{M}}^{EF}(q) \cup \text{update}$ ;
14  | if  $\text{update} \neq \text{false}$  then
15    | for each  $q' \in \text{Pre}_B(q)$  do
16      | | Send EF message update to Labelling $q'$ ;
17 while true;
18 for each  $q' \in \text{Pre}_B(q)$  do
19   | Send EX message  $\mathcal{L}_{\mathcal{M}}^{EF}(q)$  to Labelling $q'$ ;
20  $\mathcal{L}_{\mathcal{M}}^{EXEF}(q) = \text{false}$ ;
21 do
22   if all other Labelling $q$  blocked in receiving then
23     | Notify other Labelling $q$  stopping receiving
24     | and breaking;
25     | Break;
26   | Wait until an EX message arrives;
27   | Receive a Set message ex_message;
28   |  $\mathcal{L}_{\mathcal{M}}^{EXEF}(q) =$ 
29     |  $\mathcal{L}_{\mathcal{M}}^{EXEF}(q) \cup (\text{Pre}_M(\text{ex\_message}) \cap \mathcal{L}(q))$ ;
29 while true;
30 Return  $\mathcal{L}_{\mathcal{M}}^{EXEF}(q)$ ;

```

---

In Algorithm 4, lines 5-20 collaboratively compute the **EF** with other workers. At each stage, the algorithm continuously processes incoming messages from workers associated with its successors in the Büchi property automaton, updating its own set of labeled fair states. Upon receiving a message containing the newly labeled fair states from one of the successors, the algorithm labels state  $q$  with the predecessors of those fair states in the Kripke structure, ensuring that these predecessors also satisfy the state  $q$ , as shown in lines 12-13. If the set of newly discovered states, denoted as *update*, is not empty, the algorithm sends them to its predecessors as indicated by lines 15-16 of the code. Lines 21-30 compute the **EX** in the same way. After **EX** is computed, the result is returned to Algorithm 3. Before waiting for new messages, the algorithm checks whether all other workers are also waiting for incoming messages. If so, it indicates that there are no more messages being sent, and the worker notifies others to stop receiving messages and exit.

---

**Algorithm 5:** Increase Parallelism

---

**Input:** *desired\_parallelism*, *LTL\_original*

**Output:** an automaton with desired size

```

1  $LTL = LTL\_original$ ;
2  $automaton = \text{generate\_automaton\_from\_ltl}(\neg LTL)$ ;
3 while  $automaton.size < \text{desired\_parallelism}$  do
4   |  $R_s \leftarrow \text{select a sub relation of transition}$ ;
5   |  $R_{sr} \leftarrow \text{replace primed states with X operator in } R_s$ ;
6   |  $LTL_{sr} \leftarrow \bigwedge_{r \in R_{sr}} r$ ;
7   |  $LTL = \mathbf{G}(LTL_{sr}) \rightarrow LTL$ ;
8   |  $automaton =$ 
9     |  $\text{generate\_automaton\_from\_ltl}(\neg LTL)$ ;
9 Return  $automaton$ ;

```

---

### C. Optimizations

1) *Increase parallelism:* According to Algorithm 3, the number of workers is set equal to the number of states of the automaton. However, it is worth noting that the size of the automaton exhibits exponential worst-case complexity relative to the size of the formula itself [18]. Despite this exponential complexity, it does not pose a disadvantage for our approach. Despite the exponential worst-case complexity, our experiments with the algorithms described in [18] have demonstrated that they often yield automaton that are smaller in size. Nevertheless, to overcome this limitation, we have developed a solution that involves expanding the size of the automaton beyond the number of states in the original automaton. This enhancement improves the parallelism of our algorithm. Intuitively, by augmenting the complexity of the LTL formula, we can also amplify the size of the corresponding automaton.

Usually, the transition relation  $R$  of a Kripke structure is represented as a conjunction of some sub-relations  $R_1, \dots, R_n$  such that:

$$R = \bigwedge_{i=1}^n R_i, \text{ where } R_i \text{ has the form } R_i \equiv v' = f(\bar{v}) \quad (2)$$

We can select a set of sub-relations, denoted as  $R_s = \{R_i | R_i \text{ is selected}\}$ , and incorporated them into the LTL formula, which has led to an increase in its size. However, a problem arises due to the presence of primed variables  $v'$  in  $R_i$ , as these are not valid in LTL expressions. To address this issue, we need to first replace the primed variables  $v'$  with the LTL next operator **X**  $v$ , resulting in a new set of sub-relations denoted as  $R_{sr}$ . Let  $LTL_{sr} = \bigwedge_{r \in R_{sr}} r$ . Consequently, the new LTL formula will be formulated as follows:

$$LTL_{new} = (\mathbf{G} LTL_{sr}) \rightarrow LTL_{original} \quad (3)$$

To establish the relationship between the conjunction of the selected relations and the original LTL, an implication is used. This implies that if a model satisfies the selected transition relations globally, then  $LTL_{new}$  will be equisatisfiable with  $LTL_{original}$ .

Algorithm 5 outlines the procedure for increasing parallelism. It begins by checking if the size of the generated automaton is smaller than the desired parallelism. If it is, another sub-relation is added to construct a new LTL formula, and this process is repeated until the size reaches or exceeds the desired value.

2) *Load balancing:* As described in Algorithm 2, the workers collaboratively calculate the fair model state labeling function using a backward BFS approach. Generally, parallelizing a BFS-based algorithm provides inherent load balancing advantages. However, it is inevitable that load imbalance may occur in parallel algorithms due

to various factors. For example, in our approach, if only a few threads are performing BDD computation while other threads are blocked, waiting for incoming messages, load imbalance may arise.

To achieve load balancing in a scenario where some workers are blocked, waiting for incoming messages, the work-stealing strategy [19] allows these workers to assist other workers by stealing BDD operations and adding them to their own queue. Instead of developing a proprietary system to support this work-stealing approach, we utilize Sylvan [20], which is a multi-core BDD library. In Sylvan, the work-stealing strategy is employed to facilitate the stealing of BDD operation tasks from other workers when a worker becomes blocked. By leveraging the capabilities of Sylvan, the BDD operations in lines 12-13 and 28 are inputted into the library for effective load balancing.

## V. EXPERIMENTAL RESULTS

### A. Experiment Setup

We implemented the proposed property-driven parallel algorithm using Sylvan for BDD operations and ltl2tgba for automaton generation. The artifact is available at [21]. Experiments were conducted on a 32-core AMD EPYC 7532 machine with 128GB RAM.

The available benchmarks of model checking are mostly focused on invariant properties, whereas our approach focuses on LTL. We evaluated our method using cases from the examples provided in the NuSMV tool. All of the models and properties were represented in SMV format. The key parameters of the cases are outlined in Table III, where the column *Hold* signifies whether the model checking problem holds, *Variables* represents the count of boolean state variables, and *Assignments* indicates the number of next-state variable assignments, as illustrated in Equation (2).

### B. General Observation

**Baseline** Table I presents the experimental results obtained from evaluating our proposed property-driven algorithm. The sequential property-driven algorithm, outlined in Algorithm 2 and denoted as PSMC, is employed as the baseline for parallel performance evaluation. To establish its comparability to state-of-the-art symbolic model checkers, the column *NuSMV* showcases the performance data of NuSMV on the cases. It becomes evident that the baseline algorithm outperforms NuSMV in nearly all cases.

**Scalability** The performance of the parallel version of PSMC on various numbers of cores, denoted as  $PPSMC_{XX}$ , is detailed in Table I. In particular, PPSMC achieves a parallel speedup ranging from 2.81 to 17.19 times compared to PSMC in a 32-core environment, with an average speedup of 8.18 times. Figure 2 further illustrates the speedup of PPSMC at various core counts on average. It is evident that as the number of cores increases, there is no significant slowdown in the upward trend. This observation validates the near-linear scalability of PPSMC, affirming its efficient handling of larger workloads.

**Increase parallelism** Table II presents the sizes of automaton generated using different numbers of sub-relations on specific models. As the number of sub-relations increases, more intricate LTL formulas are constructed, leading to larger automaton. This size increase is instrumental in enhancing the parallelism of our method. The results show that our method effectively improves parallelism by constructing complex automaton through sub-relation selection.

**Load balancing** In Table I, we also presents the runtime of PPSMC without load balancing. The result shows that load balancing leads to a significant improvement in performance, with an average improvement of 61%. This result clearly demonstrates the effectiveness of load balancing in our method.

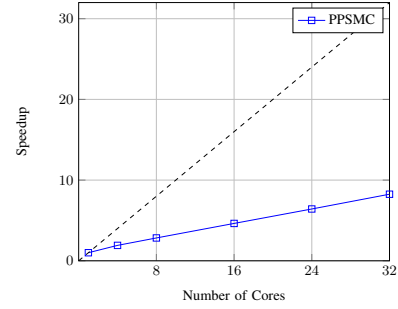


Fig. 2: PPSMC Average Speedups  
TABLE II: Parallelism Increase

| Model        | $ R_s $ | 0  | 1  | 2  | 3  | 4  |
|--------------|---------|----|----|----|----|----|
| phils p1     | $ Q_B $ | 2  | 5  | 9  | 17 | 33 |
| elevator     | $ Q_B $ | 10 | 18 | 35 | 69 |    |
| syncarb      | $ Q_B $ | 3  | 7  | 13 | 25 | 49 |
| prod-cons-p0 | $ Q_B $ | 4  | 19 | 39 | 71 |    |

### C. Comparison

**Compare to Sylvan** To our knowledge, the only existing parallel symbolic model checking approach for full LTL relies solely on parallel BDD libraries like Sylvan [20]. The advantage of this approach is that the high-level model checking algorithm remains unchanged, and all that needs to be done is to invoke the Sylvan library. The strength of our approach lies in an additional level of property-driven parallelization, which not only enhances the overall level of parallelism but also contributes to improved load balancing. We evaluated Algorithm 1 implemented by Sylvan in a 32-core environment. The results are presented in the column labeled *Sylvan<sub>32</sub>* of Table III. It is evident from the column labeled *Improv.*, when comparing the column *Sylvan<sub>32</sub>* against the column *PPSMC<sub>32</sub>*, that PPSMC consistently outperforms parallel Sylvan SMC in most cases. The improvement reaches a maximum of up to 134%, with an average improvement of 33.1%.

**Compare to LTSmin** We also evaluated LTSmin [22], which is a state-of-the-art parallel explicit model checker. We input SMV models into LTSmin through the PINS interface since LTSmin does not support SMV frontend. The results are shown in the column *LTSmin<sub>32</sub>*. It is apparent that LTSmin has encountered difficulties in solving numerous instances, highlighting the superior scalability of symbolic model checking over explicit methods.

**Compare to NuXmv** We conducted an evaluation of the k-liveness algorithm [23] implemented in NuXmv [24], a variant of the IC3 algorithm designed for checking full LTL properties. The experimental results are presented in Table III. While it may be unfair to directly compare PPSMC<sub>32</sub> with NuXmv, it's noteworthy that PPSMC<sub>32</sub> outperforms NuXmv in ten out of twenty cases. In particular, in the case of *cuhanoi10ro*, NuXmv experienced a timeout, whereas BDD-based methods successfully solved it. Despite the prevalent advantages of SAT-based methods in recent years, certain problem classes still rely on BDD-based techniques as the sole method for achieving automated solutions. BDD-based techniques are often considered as part of a diverse range of methods.

## VI. RELATED WORK

*Explicit parallel model checking* individually stores and processes each state encountered during the state space exploration. This approach facilitates the design and implementation of various parallel algorithms. In the case of DFS, the existing research mainly revolves around variants of the classical nested DFS, where a back edge indicates the presence of a loop [25]. On the other hand, in BFS, loop detection is achieved using methods such as back-level edges [17]. The issue with explicit parallel model checking is that, despite the involvement of additional resources

TABLE I: PPSMC Experimental Result

| Model           | NuSMV  | PSMC   | PPSMC <sub>4</sub> | PPSMC <sub>8</sub> | PPSMC <sub>16</sub> | PPSMC <sub>32</sub> | Speedup | W/O LB | LB Improv. |
|-----------------|--------|--------|--------------------|--------------------|---------------------|---------------------|---------|--------|------------|
| abp8 p0         | 6.28   | 1.13   | 0.67               | 0.42               | 0.29                | 0.16                | x7.06   | 0.28   | 75%        |
| abp8 p1         | 7.86   | 2.01   | 1.39               | 0.83               | 0.52                | 0.29                | x6.93   | 0.59   | 103%       |
| prod-cons p0    | 36.17  | 4.03   | 2.97               | 1.59               | 0.89                | 0.46                | x8.76   | 0.56   | 22%        |
| prod-cons p1    | 18.34  | 7.21   | 4.79               | 2.57               | 1.31                | 0.79                | x9.13   | 1.21   | 53%        |
| prod-cons p2    | 5.57   | 2.35   | 1.82               | 1.08               | 0.54                | 0.31                | x7.58   | 0.44   | 42%        |
| bc57-sensors p0 | 32.42  | 21.58  | 14.51              | 9.1                | 5.73                | 3.01                | x7.17   | 4.59   | 52%        |
| bc57-sensors p1 | 560.11 | 189.19 | 91.26              | 55.94              | 32.39               | 19.87               | x9.52   | 33.13  | 67%        |
| dme5            | 15.88  | 23.42  | 10.83              | 7.83               | 4.74                | 3.18                | x7.36   | 5.45   | 71%        |
| dme6            | 36.69  | 59.07  | 34.44              | 20.8               | 12.01               | 7.98                | x7.4    | 15.47  | 94%        |
| viscoherence p0 | 12.1   | 4.76   | 3.24               | 2.83               | 2.32                | 1.68                | x2.83   | 2.32   | 38%        |
| viscoherence p1 | 9.56   | 5.68   | 4.35               | 3.67               | 2.63                | 2.02                | x2.81   | 3.11   | 54%        |
| cunim1ro        | 209.44 | 306.19 | 120.21             | 69.92              | 41.45               | 24.11               | x12.7   | 34.85  | 45%        |
| cuhanoi7ro      | 2.19   | 2.78   | 1.33               | 0.78               | 0.5                 | 0.29                | x9.59   | 0.57   | 97%        |
| cuhanoi10ro     | 210.69 | 234.43 | 84.25              | 46.34              | 25.09               | 13.64               | x17.19  | 17.47  | 28%        |
| cuabq2mfro      | 173.69 | 233.59 | 154.21             | 92.04              | 54.71               | 26.74               | x8.74   | 46.31  | 73%        |
| phils p0        | 91.37  | 46.64  | 33.93              | 28.04              | 13.59               | 5.73                | x8.14   | 10.59  | 84%        |
| phils p1        | 140.18 | 36.76  | 24.09              | 18.35              | 10.31               | 5.96                | x6.17   | 8.02   | 35%        |
| msi-wtrans      | 41.36  | 21.33  | 12.15              | 6.73               | 3.72                | 1.74                | x12.26  | 2.54   | 46%        |
| syncarb         | 35.5   | 29.5   | 22.38              | 14.66              | 9.49                | 5.93                | x4.97   | 9.43   | 59%        |
| elevator        | 5.58   | 2.09   | 1.16               | 0.75               | 0.47                | 0.29                | x7.21   | 0.5    | 72%        |
| Average         |        |        |                    |                    |                     |                     | x8.18   |        | 61%        |

TABLE III: Experimental Result of Different Tools

| Model           | Hold | Variables | Assignments | NuXmv  | PPSMC <sub>32</sub> | Sylvan <sub>32</sub> | Improv. | LTSmin <sub>32</sub> |
|-----------------|------|-----------|-------------|--------|---------------------|----------------------|---------|----------------------|
| abp8 p0         | yes  | 49        | 13          | 4.32   | 0.16                | 0.26                 | 63%     | MO                   |
| abp8 p1         | yes  | 49        | 13          | 9.93   | 0.29                | 0.5                  | 72%     | MO                   |
| prod-cons p0    | no   | 28        | 14          | 1.01   | 0.46                | 0.58                 | 26%     | 1.71                 |
| prod-cons p1    | no   | 28        | 14          | 1.56   | 0.79                | 1.21                 | 53%     | 1.64                 |
| prod-cons p2    | yes  | 28        | 14          | 1.22   | 0.31                | 0.41                 | 32%     | 45.26                |
| bc57-sensors p0 | yes  | 78        | 69          | 20.42  | 3.01                | 2.18                 | -28%    | MO                   |
| bc57-sensors p1 | no   | 78        | 69          | 116.75 | 19.87               | 31.41                | 58%     | MO                   |
| dme5            | no   | 93        | 96          | 0.59   | 3.18                | 2.76                 | -13%    | TO                   |
| dme6            | no   | 111       | 115         | 0.76   | 7.98                | 8.04                 | 1%      | TO                   |
| viscoherence p0 | no   | 45        | 37          | 1.31   | 1.68                | 1.72                 | 2%      | 2.35                 |
| viscoherence p1 | no   | 45        | 37          | 1.88   | 2.02                | 1.96                 | -3%     | 3.02                 |
| cunim1ro        | yes  | 38        | 32          | 0.77   | 24.11               | 26.06                | 8%      | TO                   |
| cuhanoi7ro      | no   | 18        | 13          | 87.86  | 0.29                | 0.32                 | 10%     | 0.19                 |
| cuhanoi10ro     | no   | 24        | 19          | TO     | 13.64               | 15.61                | 14%     | 15.37                |
| cuabq2mfro      | yes  | 41        | 35          | 0.42   | 26.74               | TO                   | -       | MO                   |
| phils p0        | no   | 32        | 29          | 0.14   | 5.73                | 6.21                 | 8%      | 4.19                 |
| phils p1        | no   | 32        | 29          | 0.19   | 5.96                | 9.30                 | 56%     | 2.31                 |
| msi-wtrans      | yes  | 45        | 21          | 0.32   | 1.74                | 2.15                 | 24%     | MO                   |
| syncarb         | yes  | 90        | 60          | 2.07   | 5.93                | 12.77                | 115%    | MO                   |
| elevator        | yes  | 39        | 20          | 58.94  | 0.29                | 0.68                 | 134%    | MO                   |
| Average         |      |           |             |        |                     |                      | 33.1%   |                      |

in the parallel environment, its capability to handle large-scale problems remains limited [8].

*Symbolic parallel model checking* involves processing states as sets. However, treating states encountered during the search as sets poses a challenge in devising efficient nested fixpoint computing algorithms. As a result, current research primarily focuses on reachability analysis or safety properties. For instance, some studies have focused on verification of split-invariants for asynchronously processes communicating through shared-memory [15], and reachability analysis based on partitioned BDDs [26]. In contrast to these studies that primarily focus on safety properties or invariants, [16] proposed a symbolic algorithm for propositional  $\mu$ -calculus by distributively evaluating subformulas. However, this approach is limited in terms of parallelism due to the relatively small scale of the formulas. The work presented in [27] concentrates on a fragment of CTL utilizing partitioned BDDs and fixpoint computation, as opposed to nested fixpoint computation. Moreover, there exist symbolic parallel techniques that exclusively focus on parallelizing supporting BDD libraries [20]. Nevertheless, these approaches overlook the parallelization of the high-level model checking itself. Our previous experimental results highlight the need for further significant improvement in their speedup. In our work, we concentrate on the Kripke structure and the LTL automaton.

We propose a fully parallelized algorithm, while the approach does not incorporate any parallelization techniques.

## VII. CONCLUSION

This paper introduces a novel approach for parallel LTL symbolic model checking. Departing from product computation between the model and the automaton, our approach computes images on the automaton states in parallel. Experimental results showcase the scalability of our method, outperforming existing methods that rely solely on parallel BDD library or are based on explicit parallel method.

## ACKNOWLEDGMENT

This work was supported by the Foundation of Laboratory for Advanced Computing and Intelligence Engineering (Grant No. 2023-LYJJ-01-013), Beijing Municipal Natural Science Foundation (Grant No. 4252024), the Basic Research Projects from the Institute of Software, Chinese Academy of Sciences (Grant No. ISCAS-JCZD-202307) and the National Natural Science Foundation of China (Grant No. 62372438).

# REFERENCES

- [1] E. M. Clarke, O. Grumberg, D. Kroening, D. A. Peled, and H. Veith, *Model checking, 2nd Edition*. MIT Press, 2018.
- [2] E. M. Clarke, W. Klieber, M. Nováček, and P. Zuliani, "Model checking and the state explosion problem," in *Tools for Practical Software Verification, LASER, International Summer School 2011, Elba Island, Italy, Revised Tutorial Lectures*, ser. Lecture Notes in Computer Science, B. Meyer and M. Nordio, Eds., vol. 7682. Springer, 2011, pp. 1–30. [Online]. Available: <https://doi.org/10.1145/876638.876643>
- [3] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-guided abstraction refinement for symbolic model checking," *J. ACM*, vol. 50, pp. 752–794, 9 2003. [Online]. Available: <https://doi.org/10.1145/876638.876643>
- [4] J. R. Burch, E. M. Clarke, K. L. McMillan, D. L. Dill, and L. J. Hwang, "Symbolic model checking: 10<sup>20</sup> states and beyond," *Inf. Comput.*, vol. 98, no. 2, pp. 142–170, 1992.
- [5] A. Laarman, J. van de Pol, and M. Weber, "Multi-core Itsmin: Marrying modularity and scalability," in *NASA Formal Methods - Third International Symposium, NFM 2011, Pasadena, CA, USA, April 18-20, 2011. Proceedings*, ser. Lecture Notes in Computer Science, M. G. Bobaru, K. Havelund, G. J. Holzmann, and R. Joshi, Eds., vol. 6617. Springer, 2011, pp. 506–511. [Online]. Available: [https://doi.org/10.1007/978-3-642-20398-5\\_40](https://doi.org/10.1007/978-3-642-20398-5_40)
- [6] S. Ben-David, O. Grumberg, T. Heyman, and A. Schuster, "Scalable distributed on-the-fly symbolic model checking," *International Journal on Software Tools for Technology Transfer*, vol. 4, pp. 496–504, 2003.
- [7] E. Bartocci, R. DeFrancisco, and S. A. Smolka, "Towards a gpgpu-parallel spin model checker," in *Proceedings of the 2014 International SPIN Symposium on Model Checking of Software*, ser. SPIN 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 87–96.
- [8] J. Barnat, V. Bloemen, A. Duret-Lutz, A. Laarman, L. Petrucci, J. van de Pol, and E. Renault, "Parallel model checking algorithms for linear-time temporal logic," in *Handbook of Parallel Constraint Reasoning*, Y. Hamadi and L. Sais, Eds. Cham: Springer International Publishing, 2018, ch. 12, pp. 457–507. [Online]. Available: [https://doi.org/10.1007/978-3-319-63516-3\\_12](https://doi.org/10.1007/978-3-319-63516-3_12)
- [9] R. Greenlaw, H. J. Hoover, and W. L. Ruzzo, *Limits to Parallel Computation: P-Completeness Theory*. Oxford University Press, Inc., 1995.
- [10] C. Courcoubetis, M. Y. Vardi, P. Wolper, and M. Yannakakis, "Memory-efficient algorithms for the verification of temporal properties," *Formal Methods Syst. Des.*, vol. 1, no. 2/3, pp. 275–288, 1992.
- [11] R. E. Bryant, "Graph-based algorithms for boolean function manipulation," *IEEE Trans. Comput.*, vol. 35, pp. 677–691, 8 1986. [Online]. Available: <https://doi.org/10.1109/TC.1986.1676819>
- [12] E. A. Emerson and C. Lei, "Efficient model checking in fragments of the propositional mu-calculus (extended abstract)," in *Proceedings of the Symposium on Logic in Computer Science (LICS '86), Cambridge, Massachusetts, USA, June 16-18, 1986*. IEEE Computer Society, 1986, pp. 267–278.
- [13] S. Ben-David, T. Heyman, O. Grumberg, and A. Schuster, "Scalable distributed on-the-fly symbolic model checking," in *Formal Methods in Computer-Aided Design, Third International Conference, FMCAD 2000, Austin, Texas, USA, November 1-3, 2000, Proceedings*, ser. Lecture Notes in Computer Science, W. A. H. Jr. and S. D. Johnson, Eds., vol. 1954. Springer, 2000, pp. 390–404. [Online]. Available: [https://doi.org/10.1007/3-540-40922-X\\_24](https://doi.org/10.1007/3-540-40922-X_24)
- [14] A. James, A. Tiu, and N. Yatapanage, "PFMC: A parallel symbolic model checker for security protocol verification," in *Formal Methods and Software Engineering - 23rd International Conference on Formal Engineering Methods, ICFEM 2022, Madrid, Spain, October 24-27, 2022, Proceedings*, ser. Lecture Notes in Computer Science, A. Riesco and M. Zhang, Eds., vol. 13478. Springer, 2022, pp. 173–189. [Online]. Available: [https://doi.org/10.1007/978-3-031-17244-1\\_11](https://doi.org/10.1007/978-3-031-17244-1_11)
- [15] A. Cohen, K. S. Namjoshi, Y. Sa'ar, L. D. Zuck, and K. I. Kisiyova, "Parallelizing a symbolic compositional model-checking algorithm," in *Hardware and Software: Verification and Testing - 6th International Haifa Verification Conference, HVC 2010, Haifa, Israel, October 4-7, 2010. Revised Selected Papers*, ser. Lecture Notes in Computer Science, S. Barner, I. G. Harris, D. Kroening, and O. Raz, Eds., vol. 6504. Springer, 2010, pp. 46–59. [Online]. Available: [https://doi.org/10.1007/978-3-642-19583-9\\_9](https://doi.org/10.1007/978-3-642-19583-9_9)
- [16] O. Grumberg, T. Heyman, and A. Schuster, "Distributed symbolic model checking for  $\mu$ -calculus," *Formal Methods in System Design*, vol. 26, pp. 197–219, 2005.
- [17] M. Xie, Q. Yang, J. Zhai, and Q. Wang, "A vertex centric parallel algorithm for linear temporal logic model checking in pregel," *Journal of Parallel and Distributed Computing*, vol. 74, no. 11, pp. 3161–3174, 2014.
- [18] R. Gerth, D. A. Peled, M. Y. Vardi, and P. Wolper, "Simple on-the-fly automatic verification of linear temporal logic," in *Protocol Specification, Testing and Verification XV, Proceedings of the Fifteenth IFIP WG6.1 International Symposium on Protocol Specification, Testing and Verification, Warsaw, Poland, June 1995*, ser. IFIP Conference Proceedings, P. Dembinski and M. Sredniawa, Eds., vol. 38. Chapman & Hall, 1995, pp. 3–18.
- [19] M. Böhm and E. Speckenmeyer, "A fast parallel sat-solver — efficient workload balancing," *Annals of Mathematics and Artificial Intelligence*, vol. 17, pp. 381–400, 1996.
- [20] T. Van Dijk and J. Van de Pol, "Sylvan: multi-core framework for decision diagrams," *International Journal on Software Tools for Technology Transfer*, vol. 19, pp. 675–696, 2017.
- [21] "Ppsmc artifact," <https://github.com/gipsyh/PPSMC>.
- [22] G. Kant, A. Laarman, J. Meijer, J. van de Pol, S. Blom, and T. van Dijk, "Ltsmin: High-performance language-independent model checking," in *Tools and Algorithms for the Construction and Analysis of Systems - 21st International Conference, TACAS 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015. Proceedings*, ser. Lecture Notes in Computer Science, C. Baier and C. Tinelli, Eds., vol. 9035. Springer, 2015, pp. 692–707. [Online]. Available: [https://doi.org/10.1007/978-3-662-46681-0\\_61](https://doi.org/10.1007/978-3-662-46681-0_61)
- [23] K. Claessen and N. Sörensson, "A liveness checking algorithm that counts," in *Formal Methods in Computer-Aided Design, FMCAD 2012, Cambridge, UK, October 22-25, 2012*, G. Cabodi and S. Singh, Eds. IEEE, 2012, pp. 52–59. [Online]. Available: <https://ieeexplore.ieee.org/document/6462555/>
- [24] R. Cavada, A. Cimatti, M. Dorigatti, A. Griggio, A. Mariotti, A. Micheli, S. Mover, M. Roveri, and S. Tonetta, "The nuxmv symbolic model checker," in *Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings*, ser. Lecture Notes in Computer Science, A. Biere and R. Bloem, Eds., vol. 8559. Springer, 2014, pp. 334–342. [Online]. Available: [https://doi.org/10.1007/978-3-319-08867-9\\_22](https://doi.org/10.1007/978-3-319-08867-9_22)
- [25] A. Laarman and D. Faragó, "Improved on-the-fly livelock detection," in *NASA Formal Methods, 5th International Symposium, NFM 2013, Moffett Field, CA, USA, May 14-16, 2013. Proceedings*, ser. Lecture Notes in Computer Science, G. Brat, N. Rungta, and A. Venet, Eds., vol. 7871. Springer, 2013, pp. 32–47. [Online]. Available: [https://doi.org/10.1007/978-3-642-38088-4\\_3](https://doi.org/10.1007/978-3-642-38088-4_3)
- [26] T. Heyman, D. Geist, O. Grumberg, and A. Schuster, "Achieving scalability in parallel reachability analysis of very large circuits," in *Computer Aided Verification, 12th International Conference, CAV 2000, Chicago, IL, USA, July 15-19, 2000, Proceedings*, ser. Lecture Notes in Computer Science, E. A. Emerson and A. P. Sistla, Eds., vol. 1855. Springer, 2000, pp. 20–35. [Online]. Available: [https://doi.org/10.1007/10722167\\_6](https://doi.org/10.1007/10722167_6)
- [27] S. K. Iyer, D. Sahoo, C. Stangier, A. Narayan, and J. Jain, "Improved symbolic verification using partitioning techniques," in *Correct Hardware Design and Verification Methods, 12th IFIP WG 10.5 Advanced Research Working Conference, CHARME 2003, L'Aquila, Italy, October 21-24, 2003, Proceedings*, ser. Lecture Notes in Computer Science, D. Geist and E. Tronci, Eds., vol. 2860. Springer, 2003, pp. 410–424. [Online]. Available: [https://doi.org/10.1007/978-3-540-39724-3\\_35](https://doi.org/10.1007/978-3-540-39724-3_35)